



Project Report

DealForge: An Agentic AI-Powered CRM System

Prepared by: Amira Aljuaid, Shahad Alharbi, Asrar Fallatah

Table of Contents

1. Executive Summary.....	4
2. Introduction	4
3. Problem Statement	5
4. Project Objectives	5
5. Proposed Solution	6
6. System Architecture.....	6
Architecture Diagram	7
6.1 User Interaction Layer	7
6.2 Agent Orchestration Layer	8
6.3 CRM Operations Layer.....	8
6.4 Analytics and Reporting Layer.....	8
6.5 Governance and Approval Layer.....	9
7. Agent design	9
7.1 Reasoning Agent	9
7.2 Reporting Agent.....	10
7.3 Agent Collaboration Workflow	10
8. Database Design.....	11
ERD Diagram	11
9. Methodology.....	13
10. Key Features	14
11. Technologies Used.....	15
12. Security and Governance.....	16
13. System Monitoring and Observability.....	17
14. Testing and Validation	17
Functional Testing	17
Integration Testing.....	17
Validation Testing.....	18
Governance Testing	18
Observability Testing.....	18
15. Results and Achievements	18

16. Limitations.....	19
17. Future Work.....	19
19. Conclusion.....	20
References.....	20
Appendix A – User Guide	21
Appendix B – Deployment Guide	22

1. Executive Summary

DealForge is an Agentic AI-powered CRM system that automates sales operations through natural language interactions. The system enables users to create, update, retrieve, and manage CRM records conversationally while maintaining governance through approval workflows and audit logging. By integrating Large Language Models (LLMs), autonomous agents, CRM operation tools, DealForge reduces manual data entry, improves data consistency, and provides real-time business insights. The platform demonstrates how Agentic AI can enhance productivity, streamline sales processes, and transform traditional CRM systems into intelligent business assistants.

2. Introduction

Customer Relationship Management (CRM) systems play a critical role in managing customer interactions, sales opportunities, and business growth. However, most traditional CRM platforms rely heavily on manual data entry and repetitive administrative tasks,

leading to inefficiencies, delayed updates, and inconsistent records

DealForge addresses these challenges by introducing an Agentic AI-powered CRM system capable of understanding natural language requests, autonomously executing CRM operations, and maintaining accurate customer data in real time. By combining Large Language Models (LLMs), intelligent agents, and governance mechanisms, DealForge transforms conversational interactions into actionable CRM workflows.



3. Problem Statement

Organizations using traditional CRM systems commonly face several operational challenges:

- Extensive manual data entry and record maintenance.
- Time-consuming updates of leads, contacts, companies, and deals.
- Human errors leading to inconsistent CRM data.
- Delays between customer interactions and CRM updates.
- Limited visibility into sales performance and pipeline health.
- Low user adoption due to the complexity of CRM interfaces.

These challenges reduce productivity and prevent sales teams from focusing on high-value customer engagement activities.

4. Project Objectives

The primary objectives of DealForge are:

- Automate CRM operations through natural language interactions.
- Convert user conversations into structured CRM actions.
- Reduce manual effort and administrative overhead.
- Improve CRM data accuracy and consistency.
- Enable real-time CRM updates.
- Provide governance and approval mechanisms for sensitive operations.
- Generate intelligent sales insights and analytics.
- Enhance user productivity through an AI-driven experience.



5. Proposed Solution

DealForge is an Agentic AI CRM system that acts as an intelligent intermediary between users and CRM databases.

The platform enables users to interact with CRM data using natural language while autonomous AI agents interpret requests, execute actions, and generate analytics.

The system supports:

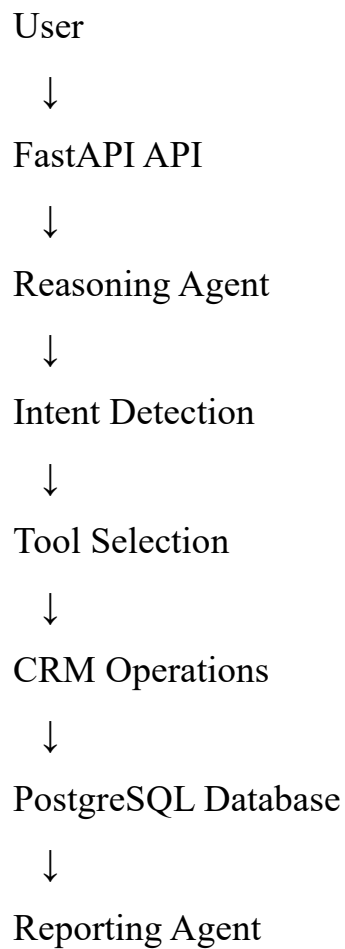
- Lead creation and management.
- Contact, company and Campaign management.
- Deal tracking and pipeline updates.
- Task generation and follow-up management.
- Intelligent reporting and KPI analysis.
- Human approval workflows for critical modifications.

6. System Architecture

DealForge follows a multi-layered agentic architecture designed to ensure scalability, reliability, and governance. The system consists of several interconnected layers that collaboratively process user requests and execute CRM operations.



Architecture Diagram



6.1 User Interaction Layer

Responsible for receiving user requests through conversational interfaces.

Functions:

- Captures natural language requests.
- Sends requests to the orchestration layer.
- Displays responses and approval requests.



6.2 Agent Orchestration Layer

The central intelligence layer responsible for understanding user intentions and coordinating workflow execution.

Functions:

- Intent classification.
- Entity extraction.
- Workflow planning.
- Agent routing.
- Context management.

6.3 CRM Operations Layer

Executes CRM-related actions through specialized tools.

Supported operations:

- Create, update, and retrieve leads.
- Manage contacts and companies.
- Update deal stages.
- Generate tasks.
- Perform CRM searches.

6.4 Analytics and Reporting Layer

Provides business intelligence capabilities.

Functions:

- Dashboard generation.
- Sales pipeline analysis.
- Lead conversion analysis.
- KPI calculation.
- Risk assessment.



- AI-generated recommendations.

6.5 Governance and Approval Layer

Ensures safe execution of CRM operations.

Functions:

- Approval workflows.
- Pending update management.
- Audit logging.
- Security validation.
- Human-in-the-loop control.

7. Agent design

DealForge follows a specialized multi-agent architecture where each agent is responsible for a specific business function. The agents collaborate through the orchestration layer to execute CRM workflows efficiently and accurately.

7.1 Reasoning Agent

The Reasoning Agent acts as the core decision-making component of the system.

Responsibilities:

- Understand user requests written in natural language.
- Detect user intent.
- Extract CRM entities and business information.
- Determine the required workflow.
- Select the appropriate CRM tools.
- Coordinate execution steps.



Examples:

- Create a new lead from conversation
- Update deal information based on user input
- Retrieve company details from CRM
- Generate follow-up tasks automatically

7.2 Reporting Agent

The Reporting Agent is responsible for business intelligence and analytics generation.

Responsibilities:

- Analyze CRM records.
- Calculate sales KPIs.
- Generate pipeline reports.
- Evaluate lead conversion performance.
- Identify sales trends and risks.
- Produce actionable recommendations.

Examples:

- Sales performance summaries.
- Lead conversion reports.
- Pipeline health analysis.
- Revenue opportunity insights.

7.3 Agent Collaboration Workflow

The interaction between agents follows the sequence below:

1. User submits a request through the conversational interface.

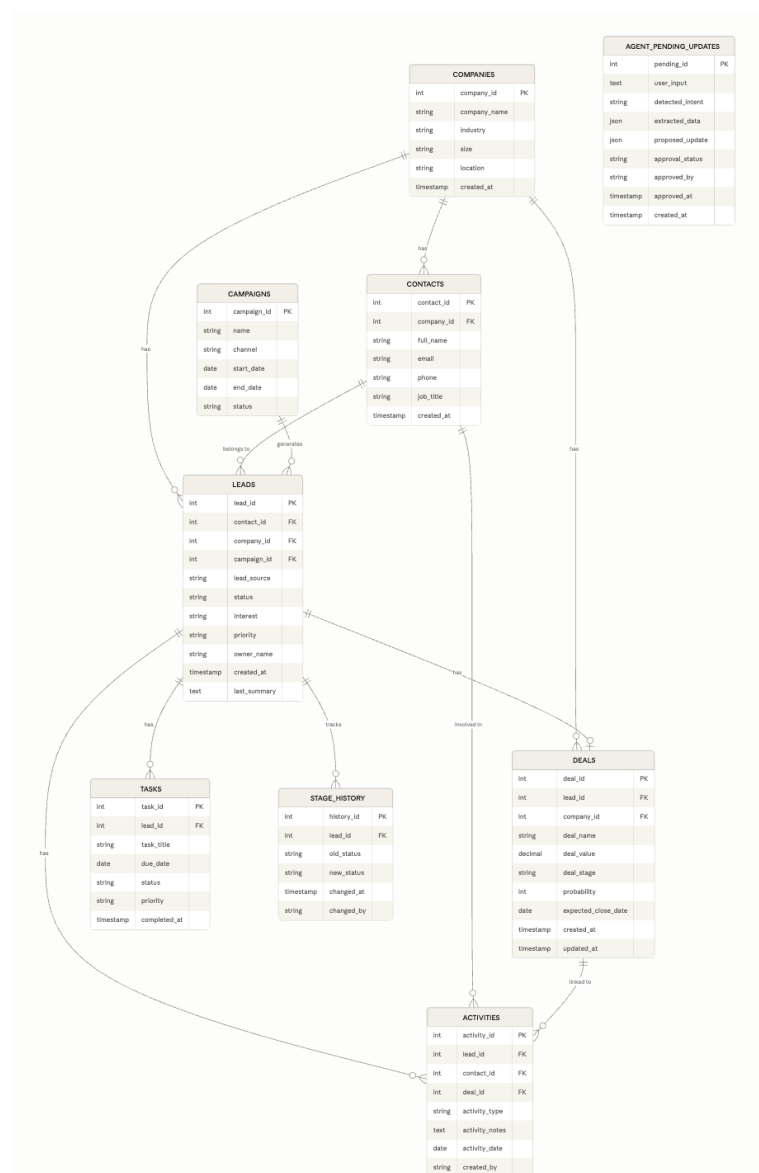


2. The Reasoning Agent analyzes the request and determines the required action.
3. CRM tools execute the requested operation.
4. If analytics are required, the Reporting Agent processes CRM data.
5. Results are returned to the user.
6. All actions are logged for governance and traceability.

This separation of responsibilities improves scalability, maintainability, and future extensibility of the platform.

8. Database Design

ERD Diagram



Key tables include:

- Leads
 - lead_id, contact_id, company_id, campaign_id, lead_source, status, interest, priority, owner_name, created_at, last_summary
- Contacts
 - contact_id, company_id, full_name, email, phone, job_title, created_at
- Companies
 - company_id, company_name, industry, size, location, created_at
- Deals
 - deal_id, lead_id, company_id, deal_name, deal_value, deal_stage, probability, expected_close_date, created_at, updated_at
- Campaigns
 - campaign_id, name, channel, start_date, end_date, status
- Activities
 - activity_id, lead_id, contact_id, deal_id, activity_type, activity_notes, activity_date, created_by
- Tasks
 - task_id, lead_id, task_title, due_date, status, priority, completed_at
- Stage History
 - history_id, lead_id, old_status, new_status, changed_at, changed_by



- Agent Pending Updates
 - pending_id, user_input, detected_intent, extracted_data, proposed_update, approval_status, approved_by, approved_at, created_at
- Long Term Memory
 - FAISS vector store persisting conversation history and CRM context for AI retrieval (RAG)

9. Methodology

The system follows an agentic execution pipeline:

Step 1: Input Processing

User requests are received in natural language.

Step 2: Intent Detection

The AI identifies the requested action.

Examples:

- Create lead
- Update company
- Generate report
- Retrieve contact details

Step 3: Entity Extraction

Relevant entities are extracted from the conversation.

Examples:

- Lead name
- Company name
- Contact details
- Deal information



Step 4: Workflow Planning

The orchestration layer determines the required tools and execution sequence.

Step 5: Validation and Governance

Sensitive actions are validated and may require approval.

Step 6: Execution

CRM actions are executed against the database.

Step 7: Reporting and Logging

Results are recorded for traceability and analytics.

10. Key Features

- **Agentic CRM Automation**
Autonomous execution of CRM workflows.
- **Natural Language CRM Interface**
Users interact with CRM using conversational language.
- **Intelligent Entity Extraction**
Automatic extraction of structured CRM data.
- **Real-Time CRM Updates**
Instant synchronization of CRM records.
- **Approval Workflow**
Human approval for critical CRM changes.
- **Reporting and Analytics**
Automated KPI generation and business insights.
- **Auditability**
Comprehensive activity logging and traceability.



11. Technologies Used

Artificial Intelligence

- OpenAI GPT Model
- LangChain
- Agent-based workflow architecture
- LangGraph (agent workflow orchestration)
- FAISS (vector store for long-term memory and RAG)

Backend

- Python
- FastAPI

Database

- PostgreSQL
- SQLAlchemy ORM

Monitoring and Observability

- LangSmith
- Execution tracing
- Tool-level monitoring

Data Processing

- BeautifulSoup
- Requests
- JSON processing
- Plotly (interactive dashboard charts)



- ReportLab (PDF report generation)

Development Tools

- GitHub
- VS Code
- Postman

Frontend

- React 19 (component-based UI framework)
- Vite (frontend build tool and dev server)
- Lucide React (icon library)

12. Security and Governance

Security and reliability were key design considerations.

Implemented mechanisms include:

- **Approval Workflow**
Critical updates require user confirmation.
- **Audit Logging**
Records all CRM actions and system decisions.
- **Input Validation**
Prevents malformed or unsafe operations.
- **Traceability**
All agent actions can be monitored through LangSmith.



13. System Monitoring and Observability

The system incorporates monitoring and observability capabilities through LangSmith. Agent executions, tool invocations, workflow decisions, and execution traces are recorded to support debugging, performance evaluation, and system reliability.

14. Testing and Validation

The DealForge platform was evaluated through multiple testing phases to ensure functionality, reliability, and accuracy.

Functional Testing

Functional testing verified that all CRM operations behaved as expected.

Tested functions included:

- Lead creation
- Lead updates
- Contact management
- Company management
- Deal management
- Task generation
- Report generation

Integration Testing

Integration testing validated communication between:

- User Interface Layer
- Agent Orchestration Layer
- CRM Operations Layer



- Database Layer

Validation Testing

Validation testing ensured that extracted entities and generated CRM actions accurately reflected user intentions.

The system was evaluated using different natural language CRM requests and demonstrated successful interpretation and execution of user commands.

Governance Testing

Approval workflows were tested to confirm that sensitive operations required human authorization before execution.

Observability Testing

LangSmith tracing was used to monitor agent execution paths, tool usage, and workflow performance for debugging and validation purposes.

15. Results and Achievements

The implementation of DealForge demonstrated several benefits:

- Significant reduction in manual CRM data entry.
- Faster processing of sales-related operations.
- Improved consistency of CRM records.
- Real-time visibility into sales pipelines.
- Intelligent business reporting and recommendations.
- Enhanced user productivity through conversational workflows.



16. Limitations

Despite its capabilities, the current implementation has several limitations:

- The system currently supports a limited number of CRM workflows.
- Large Language Model responses may occasionally require human verification.
- The platform currently operates on a single CRM database and does not yet support enterprise CRM integrations.
- Advanced predictive analytics and forecasting capabilities are not fully implemented.
- System performance may vary depending on the complexity of user requests and external service availability.

17. Future Work

Future enhancements may include:

- Multi-agent collaborative workflows.
- Predictive sales forecasting.
- Automated lead scoring.
- Email and WhatsApp integration.
- Enterprise CRM integrations (Salesforce, HubSpot, Zoho).



18. Conclusion

DealForge demonstrates how Agentic AI can transform traditional CRM systems into intelligent, autonomous business platforms. By combining natural language understanding, workflow automation, reporting intelligence, and governance controls, the system significantly reduces manual effort while improving CRM accuracy and operational efficiency. The project successfully validates the feasibility of conversational CRM management and establishes a foundation for future AI-driven enterprise solutions.

References

- [1] OpenAI. (2024). OpenAI API Documentation. Retrieved from <https://platform.openai.com/docs>
- [2] LangChain. (2024). LangChain Documentation. Retrieved from <https://python.langchain.com/docs>
- [3] LangGraph. (2024). LangGraph Documentation – Building Stateful Multi-Actor Applications. Retrieved from <https://langchain-ai.github.io/langgraph>
- [4] Tiangolo, S. (2024). FastAPI Documentation. Retrieved from <https://fastapi.tiangolo.com>
- [5] PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. Retrieved from <https://www.postgresql.org/docs>
- [6] LangSmith. (2024). LangSmith Documentation – Observability for LLM Applications. Retrieved from <https://docs.smith.langchain.com>
- [7] Meta AI Research. (2024). FAISS: A Library for Efficient Similarity Search. Retrieved from <https://faiss.ai>



[8] Wang, L., et al. (2023). A Survey on Large Language Model based Autonomous Agents. arXiv preprint arXiv:2308.11432.

Appendix A – User Guide

Creating a Lead

Users can create a new lead by submitting a natural language request such as:

"Create a lead named Ahmed from ABC Company interested in AI solutions."

The system extracts the required information, generates a proposed CRM update, and requests approval before saving the record.

Updating CRM Records

Users may update leads, contacts, companies, and deals through conversational commands.

Example:

"Update Ahmed's lead status to Qualified."

The system validates the request and applies the update after approval.

Managing Pending Approvals

All critical CRM modifications are stored as pending updates.

Users can:

- Review proposed changes.
- Approve modifications.
- Reject modifications.



- View approval history.

Generating Reports

Users can request reports using natural language.

Examples:

- Generate a sales pipeline report.
- Show lead conversion statistics.
- Provide deal performance insights.

The Reporting Agent automatically analyzes CRM data and generates the requested report.

Appendix B – Deployment Guide

System Requirements

- Python 3.11 or later
- PostgreSQL Database
- OpenAI API Key
- Internet Connection
- Minimum 8 GB RAM

Installation Steps

1. Clone the project repository.
2. Install project dependencies using pip.
3. Configure environment variables.
4. Create the PostgreSQL database.
5. Execute database migrations.
6. Start the FastAPI application.



Environment Configuration

Required variables include:

- OPENAI_API_KEY
- DATABASE_URL
- LANGSMITH_API_KEY

Running the Application

Start the backend server:

```
uvicorn main:app --reload
```

The application will then be available through the configured API endpoint.

